

Attention is All You Need

Learn about and reproduce the paper

CANOPY COURSEBOOK

Contents

1. Paper orientation and Transformer fundamentals

- Paper thesis and sequence transduction setting
- Transformer encoder-decoder overview
- Why remove recurrence and convolutions

2. Core Transformer components

- Positional encoding and autoregressive masking
- Scaled dot-product attention
- Multi-head attention tensor operations
- Transformer block implementation
- Transformer mechanics check

3. Training the translation model to match the paper

- Length-based batching and training throughput
- Optimizer schedule, label smoothing, and dropout
- WMT data and subword preprocessing
- Translation decoding and checkpoint averaging lab

4. Experiments, ablations, and reproduction check

- Constituency parsing reproduction
- Transformer variant ablation study
- Translation quality and generalization
- Reproduce paper experiments check

MODULE 1

Paper orientation and Transformer fundamentals

Paper thesis and sequence transduction setting

Frame the paper's core claim: replace recurrent and convolutional sequence transduction with an attention-only encoder-decoder for translation and related structured prediction tasks.

Objective

Understand the paper's central claim: for sequence transduction tasks like translation, an encoder-decoder can be built using attention alone, without recurrent or convolutional layers.[dcf1f3ae-6f31-428f-a5bf-4cdbac135368]

Core idea

Traditional sequence-to-sequence models process tokens through recurrence or convolution, then often add attention on top. The Transformer flips that design: it uses attention as the main mechanism for representing and connecting input and output positions, while the decoder still generates tokens autoregressively.[71875727-e848-4532-a3b8-fda6a5f211ce][c66fafcc-a367-4148-8ba3-3fd023f63e5e] This matters because attention can relate distant positions in a constant number of steps, which makes long-range dependencies easier to model and training more parallelizable.[c66fafcc-a367-4148-8ba3-3fd023f63e5e]

Practical example

Suppose the input is a French sentence and the model is producing an English translation. When generating the next English word, the decoder can attend directly to the most relevant source words instead of waiting for information to flow through many recurrent steps. That direct access is especially useful when the translation depends on a word far back in the sentence.[c66fafcc-a367-4148-8ba3-3fd023f63e5e]

A common misconception is that "attention-only" means the model has no structure at all. In fact, the Transformer still has a clear encoder-decoder organization, and the decoder is still autoregressive;

what is removed is recurrence and convolution as the primary computation for sequence representations.[71875727-e848-4532-a3b8-fda6a5f211ce][c66fafcc-a367-4148-8ba3-3fd023f63e5e]

Why this thesis is important

The paper is not merely proposing another attention trick. It argues that attention can be the core computational engine for translation and related structured prediction problems, replacing older sequential building blocks while improving speed and quality.[dcf1f3ae-6f31-428f-a5bf-4cdbac135368]

Attention-only translation flow

Take an input sentence of length n . The encoder produces a sequence of representations $z=(z_1,\dots,z_n)$, one per input position, and the decoder then generates output tokens $y_1,\dots,y_m$ one at a time.[71875727-e848-4532-a3b8-fda6a5f211ce] At step t , the decoder uses the previously generated tokens as input and attends over the encoded source to decide what to emit next.[71875727-e848-4532-a3b8-fda6a5f211ce] This shows the key thesis in action: the model still maps sequences to sequences, but the internal computation relies on attention rather than recurrent or convolutional layers.[dcf1f3ae-6f31-428f-a5bf-4cdbac135368]

Check your understanding

1. Which statement best captures the Transformer thesis in this lesson?

- It replaces recurrence and convolution with attention as the main mechanism for sequence transduction.
- It keeps an RNN encoder-decoder and adds a larger attention module.
- It removes the need for any encoder-decoder structure.
- It only works for classification, not translation.

2. Explain in a few sentences why attention-only sequence transduction can help with long-range dependencies and training speed.

Sources: [1], [2], [3]

Transformer encoder-decoder overview

Describe the high-level stack of encoder and decoder layers, including self-attention, encoder-decoder attention, feed-forward sublayers, residuals, normalization, and autoregressive masking at a conceptual level.

Objective

Understand how a Transformer encoder-decoder processes a sequence: the encoder builds contextual representations of the input, and the decoder generates output tokens one step at a time using both its own past outputs and the encoder's output. The key idea is that the model uses attention stacks instead of recurrence or convolution.

[b1c7f27b-bcae-43da-99e9-f6a53344c14f][71875727-e848-4532-a3b8-fda6a5f211ce]

Core idea

The encoder and decoder are both built from repeated layers. In the encoder, each layer first applies multi-head self-attention so every input position can gather information from other input positions; then it applies a position-wise feed-forward network. In the decoder, each layer has the same kind of self-attention and feed-forward sublayers, plus a third sublayer that lets decoder states attend to the encoder output. Each sublayer is wrapped in a residual connection and followed by layer normalization.

[b1c7f27b-bcae-43da-99e9-f6a53344c14f][5529ff7f-00d0-4800-a4a9-4272557a4f41]

A practical way to think about the flow is: the encoder turns the whole source sentence into a set of contextual vectors; the decoder then reads those vectors while producing the target sequence left to right. Because generation is autoregressive, the decoder must not “peek” at future output tokens, so its self-attention is masked to block illegal connections.

[5f33f44e-6ed5-43ef-a5d6-1d1f280b3093][b1c7f27b-bcae-43da-99e9-f6a53344c14f]

Misconception to avoid

A common mistake is to think the decoder only looks at the encoder output. In fact, it uses two different sources of context: masked self-attention over its already generated tokens, and encoder-decoder attention over the input sequence. Both are needed for fluent, conditioned generation. [5529ff7f-00d0-4800-a4a9-4272557a4f41][5f33f44e-6ed5-43ef-a5d6-1d1f280b3093]

Tracing one decoder step

Suppose the source sentence is “The cat slept,” and the decoder has already produced “Le chat”. At the next step, the decoder first uses masked self-attention so the new position can depend on “Le” and “chat” but not on any future word. It then uses encoder-decoder attention to focus on the source words that are most relevant for the next translation choice. Finally, the feed-forward network transforms the result at that position before the model predicts the next token.

Check your understanding

1. Which sublayer is responsible for letting decoder positions look at the encoder’s output?
 - Masked self-attention in the decoder
 - Encoder-decoder attention
 - The encoder’s feed-forward network
 - Layer normalization
2. Which statements about the Transformer encoder-decoder are true? Select all that apply.
 - The encoder uses self-attention so each source position can incorporate information from other source positions.
 - The decoder’s self-attention is masked to prevent access to future target tokens.
 - Every sublayer in the stack is followed by a residual connection and layer normalization.
 - The feed-forward network mixes information across different sequence positions.
3. In a few sentences, explain why the decoder needs both masked self-attention and encoder-decoder attention.

Sources: [4], [1], [5], [6]

Why remove recurrence and convolutions

Explain how sequential recurrence and distance-sensitive convolutions limit parallelism and make long-range dependencies harder to model than self-attention.

Objective

Understand why Transformers replace recurrence and convolution with self-attention: to remove sequential bottlenecks and make long-range relationships easier to model.

Core idea

Recurrent models process tokens one after another, so each hidden state depends on the previous one. That means training and inference have an inherently sequential chain that limits parallelism [a6b83920-6918-47e7-b7c1-7008578c3b25]. Convolutional models improve parallelism, but a token can only interact directly with nearby tokens in one layer, so information from far-away positions must pass through many stacked layers; the number of operations grows with distance [c66fafcc-a367-4148-8ba3-3fd023f63e5e].

Self-attention removes both restrictions. Any token can look at any other token in a single layer, so the path between distant positions is short and does not depend on their separation [c66fafcc-a367-4148-8ba3-3fd023f63e5e]. The result is a model that is easier to parallelize and better suited to capturing long-range dependencies.

Practical example

Imagine translating a sentence where the subject appears near the beginning but the verb agreement is decided near the end. A recurrent model must carry that subject information through every intermediate step, and a convolutional model may need multiple layers before the two positions can influence each other. With self-attention, the verb position can directly attend to the subject position in one layer, making the dependency easier to learn.

Common misconception

A common misconception is that recurrence is "better" because it explicitly processes order. In fact, order does not require recurrence. Transformers recover order by adding positional information, while still keeping the computation parallel [eb6160da-8626-4241-825b-d13322ac5df6]. So the real tradeoff is not "order vs. no order," but "sequential processing vs. parallel processing with explicit position signals."

Worked example

One token reaches another

Suppose a sequence has length 8, and token 1 needs information from token 8. In a recurrent network, token 8 can be influenced by token 1 only after the signal has been passed through each intermediate time step, so the effective path length grows with sequence length [eb6160da-8626-4241-825b-d13322ac5df6]. In a convolutional model with a small kernel, the signal also moves gradually across positions, requiring multiple layers. Self-attention skips that chain: token 8 can directly weight token 1 in a single layer, which is why long-range dependencies are easier to model [c66fafcc-a367-4148-8ba3-3fd023f63e5e].

Check your understanding

1. Which statement best explains why recurrence limits parallelism during training?

- Each hidden state depends on the previous one, so later computations cannot be fully done until earlier ones finish.
- It requires attention weights to be normalized one token at a time.
- It can only represent short sequences.
- It prevents models from using any positional information.

2. Which advantages describe self-attention compared with recurrence or convolution? Select all that apply.

- It allows any position to directly interact with any other position in one layer.
- It removes the need for positional information entirely.
- It reduces the minimum number of sequential operations needed for a layer.
- It makes long-distance relationships easier to learn than in stacked convolutions.

Sources: [7], [2], [8]

MODULE 2

Core Transformer components

Positional encoding and autoregressive masking

Explain how sinusoidal position signals and causal masks inject order information and prevent illegal future-token attention in the decoder.

Objective

Understand how a Transformer decoder knows token order and why it must not look at future tokens while predicting the next one.

Core idea: attention is order-agnostic unless we add order information

Self-attention compares tokens as a set of vectors, so by itself it does not know whether one token came before or after another. Positional signals fix that by giving each position a distinct pattern that the model can combine with token embeddings. In the original Transformer, sinusoidal encodings are added to the embeddings so the same token can behave differently at different positions.

[b1c7f27b-bcae-43da-99e9-f6a53344c14f]

A second ingredient is the causal, or autoregressive, mask in the decoder. During next-token prediction, position i is allowed to attend only to positions up to i , not to later positions. The mask blocks illegal future-token connections by setting their attention scores to $-\infty$ before the softmax, so those links receive zero probability. [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093]

Practical example: why both pieces matter

Suppose the decoder is generating the sentence "The cat sat on the ____." At the blank, the model should use "The cat sat on the" and not peek at the true next word in the training target. The causal mask prevents that leak. At the same time, positional encoding helps the model distinguish "cat sat" from "sat cat," even though both contain the same words. [b1c7f27b-bcae-43da-99e9-f6a53344c14f]

Misconception to avoid

A common mistake is to think the mask alone gives the model full sense of order. It does stop cheating, but it does not tell the network which earlier token is first, second, or third. Without positional information, attention can know which tokens are present but not their sequence arrangement. [b1c7f27b-bcae-43da-99e9-f6a53344c14f]

Worked example

If the input tokens are [A, B, C], then the decoder at position 2 (the token B) may attend to positions 1 and 2, but not to position 3. With a causal mask, the attention row for position 2 has valid entries for A and B, and the score for C is suppressed to $-\infty$ before softmax, so C gets no weight. [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093]

Causal masking on a three-token sequence

Consider decoder positions for [A, B, C]. For position 1, only A is visible. For position 2, A and B are visible. For position 3, all three are visible. This triangular visibility pattern is what preserves autoregressive prediction. [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093]

Check your understanding

1. In one sentence: why do Transformers add positional encodings to token embeddings?

- To give the model information about token order that attention alone does not provide.
- To replace the need for attention layers entirely.
- To ensure the decoder can look at future tokens during training.

2. Which statements about causal masking are true? Select all that apply.

- It prevents a decoder position from attending to later positions.
- It is applied before the softmax by making illegal attention scores very small.
- It gives the model information about absolute token index by itself.
- It allows the model to cheat by using the target token it is supposed to predict.

3. Explain how positional encodings and causal masks work together in an autoregressive decoder.

Sources: [4], [6]

Scaled dot-product attention

Derive how queries, keys, and values produce attention weights, why the dot products are scaled by $\sqrt{d_k}$, and how masking fits into the softmax computation.

Objective

Learn how scaled dot-product attention turns queries, keys, and values into weighted outputs, why the dot products are divided by $\sqrt{d_k}$, and how masking changes the softmax step.

Core idea

For each query vector q , attention compares it with every key vector k_i using a dot product. Those scores become probabilities after softmax, and the probabilities are used to mix the value vectors v_i . In compact form, the mechanism is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\text{top}}}{\sqrt{d_k}}\right)V$$

The dot products measure how well each key matches the query, so a larger score means that value gets more influence in the output. The $\sqrt{d_k}$ scaling keeps scores from growing too large when the key/query dimension increases, which helps softmax stay sensitive instead of becoming overly peaked.

Practical example

Suppose one word in a sentence is trying to decide what context matters most. If its query is very similar to the key for a nearby pronoun, the attention weight on that pronoun becomes high, so the output vector is influenced strongly by the pronoun's value. If a different key is less relevant, its weight is lower, and its value contributes less. This is why attention can combine information from different positions in a sequence.

Why scaling matters

Without scaling, larger d_k can make dot products have larger variance, which can push the softmax toward near-0/near-1 outputs too early. Scaling by $\sqrt{d_k}$ keeps the score magnitudes in a range where the model can still compare alternatives smoothly.

Masking in the softmax

Sometimes not every key is allowed to participate. In decoder self-attention, later positions must not look ahead to future tokens. The usual trick is to add a mask before softmax: illegal positions get $-\infty$, so after softmax they receive probability 0. This preserves causality while keeping the computation inside the same attention formula. [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093]

A tiny attention calculation

Take one query and three keys with raw similarity scores $[2, 1, 0]$. If we scale by $\sqrt{d_k}=2$, the scores become $[1, 0.5, 0]$. Softmax then turns them into a distribution that still favors the first key, but not so extremely that the others vanish. The output is the weighted sum of the three value vectors, using those probabilities as weights. This illustrates the full path from similarity scores to an attention output. [5529ff7f-00d0-4800-a4a9-4272557a4f41]

Check your understanding

1. If a query is most similar to the second key, what happens in scaled dot-product attention?

- The second value gets the largest weight in the output.
- All values get exactly equal weight.
- The second key is removed before softmax.
- The query vector is replaced by the second key.

2. Which statements about scaling and masking in attention are correct?

- Dividing by $\sqrt{d_k}$ helps keep softmax from becoming overly peaked as the key dimension grows.
- Masking illegal positions by setting their logits to $-\infty$ makes their softmax probability zero.
- Scaling by $\sqrt{d_k}$ changes the dimension of the keys and queries.
- Masking is only used during training and is unnecessary at inference time.

3. Explain, in a few sentences, how a mask changes the attention computation without changing the overall attention formula.

Sources: [5], [6]

Multi-head attention tensor operations

Implement parallel attention heads with learned projections, concatenation, and output projection while preserving the model's shared `d_model` interface.

Objective

Learn how multi-head attention keeps the same shared `d_model` interface while splitting the work across several attention heads. Each head learns its own query, key, and value projections, processes a smaller subspace, and then the head outputs are combined back into one representation. The key idea is that the model can look at different relationships in parallel instead of forcing one attention pattern to do everything at once [5529ff7f-00d0-4800-a4a9-4272557a4f41].

Core idea

Start with tensors shaped like $(batch, seq_len, d_model)$. For h heads, we usually split the last dimension into h smaller chunks of size $d_k = d_v = d_model / h$. Each head applies its own learned linear projections to form queries, keys, and values, runs attention in that smaller space, and produces an output of shape $(batch, seq_len, d_v)$. Those head outputs are concatenated along the feature dimension and projected one last time back to $(batch, seq_len, d_model)$ so the rest of the network can keep using the same width [5529ff7f-00d0-4800-a4a9-4272557a4f41].

A useful mental model is: **project** $\rightarrow$ **attend in parallel** $\rightarrow$ **concatenate** $\rightarrow$ **project back**. The projections are what let different heads specialize. Without them, every head would see the same representation and you would lose most of the benefit of parallel heads [5529ff7f-00d0-4800-a4a9-4272557a4f41].

Guided walkthrough

Suppose the input has shape (B, T, d_model) . First, compute three learned linear maps for queries, keys, and values. Then reshape or split the projected tensors so that the head dimension is explicit, giving something like (B, h, T, d_k) for queries and keys and (B, h, T, d_v) for values. This layout makes it easy to perform the same attention calculation for every head at once.

Inside each head, attention scores compare every query position to every key position. After softmax, the weights mix the values into a head-specific output. The important tensor trick is that the sequence and batch dimensions stay aligned while the feature dimension is divided across heads.

After all heads finish, concatenate the head outputs along the feature axis. If each head returns (B, T, d_v) , the concatenated tensor becomes $(B, T, h * d_v)$, which is typically (B, T, d_model) . A final output projection maps that combined tensor back into the model space.

Exercise requirements

Your coding task will ask you to implement the forward pass for multi-head attention using NumPy. The function should accept batched inputs, create per-head projections, compute attention independently for each head, concatenate the head outputs, and apply the output projection. The final result must preserve the original `d_model` width so it can plug into the rest of a Transformer block.

Be careful about tensor reshaping: the head dimension should be explicit during attention, but it should disappear again after concatenation. You should also ensure that each learned projection has the correct shape for mapping from `d_model` to the per-head size.

Common mistakes

A very common bug is mixing up reshape and transpose order, which scrambles tokens across heads. Another is forgetting that the final output projection is required even after concatenation; without it, the module would no longer return the shared model width. It is also easy to accidentally use one shared projection for all heads, which removes the specialization that multi-head attention is designed to provide.

Splitting and recombining head outputs

If a tensor has shape $(2, 5, 64)$ and you use $h = 8$ heads, then each head works with $d_k = d_v = 8$ features. After attention, each head can produce an output of shape $(2, 5, 8)$. Concatenating the 8 head outputs along the last axis gives $(2, 5, 64)$ again, which can then be passed through the output projection [5529ff7f-00d0-4800-a4a9-4272557a4f41].

```
batch, seq_len, d_model = 2, 5, 64
heads = 8
per_head = d_model // heads
head_outputs = ["(2, 5, 8)"] * heads
combined = "(2, 5, 64)"
print(per_head)
print(combined)
```

Why heads need a final projection

If two heads each produce (1, 3, 4), concatenation gives (1, 3, 8). If the surrounding Transformer block expects `d_model = 8`, that already matches; if it expects a different width, the output projection is what restores the interface to the required size [5529ff7f-00d0-4800-a4a9-4272557a4f41].

```
import numpy as np
head1 = np.zeros((1, 3, 4))
head2 = np.ones((1, 3, 4))
combined = np.concatenate([head1, head2], axis=-1)
print(combined.shape)
```

Check your understanding

1. Which tensor operation is used to combine the outputs of all attention heads before the final output projection?

- Concatenate the head outputs along the feature dimension
- Average the head outputs across the head dimension
- Transpose the tensor so heads become the batch dimension
- Apply softmax across heads

2. Explain why multi-head attention usually uses separate learned projections for each head instead of sharing one projection across all heads.

Sources: [5]

Transformer block implementation

Compose attention, residual connections, layer normalization, and position-wise feed-forward networks into reusable encoder and decoder layers.

Objective

In this lesson, you'll learn how to compose the main pieces of a Transformer layer into a reusable encoder or decoder block: attention, residual connections, layer normalization, and a position-wise feed-forward network. A Transformer layer is built from stacked sub-layers, and each sub-layer is

wrapped the same way: add its output back to the input, then normalize the result. The original Transformer design uses this pattern for both encoder and decoder stacks [b1c7f27b-bcae-43da-99e9-f6a53344c14f].

Core idea

Think of a block as a small pipeline that preserves the input shape. Each sub-layer transforms the sequence, but the residual path makes it easier to optimize and helps information flow through deep stacks. In the encoder, the usual order is self-attention followed by a feed-forward network. In the decoder, there is an extra attention sub-layer that looks at the encoder output, and decoder self-attention must be masked so later positions cannot peek ahead [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093].

The feed-forward part is applied independently to every token position: the same weights are reused across the sequence, but each position is processed separately. In the original Transformer, this is described as two linear layers with a ReLU in between [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093].

Guided walkthrough

A good implementation strategy is to build one reusable helper that takes an input tensor and a sub-layer function, then returns the normalized residual result. That helper should not care whether the sub-layer is attention or feed-forward; it only needs to preserve the tensor shape.

Next, define the encoder layer by chaining self-attention and feed-forward through that helper. The decoder layer uses the same pattern, but with three sub-layers: masked self-attention, cross-attention over encoder outputs, and the feed-forward network. The important detail is that the decoder's self-attention must only use earlier positions, which is typically handled by a causal mask inside attention [5f33f44e-6ed5-43ef-a5d6-1d1f280b3093].

When you implement the feed-forward network, remember that it acts on the last dimension only. If your input has shape $(\text{batch}, \text{time}, d_{\text{model}})$, the output should keep the same batch and time dimensions while changing features internally and then returning to d_{model} .

Exercise requirements

Your coding task will likely ask you to build:

- a reusable residual-plus-layer-norm wrapper,
- an encoder layer class with self-attention and feed-forward sub-layers,
- a decoder layer class with masked self-attention, cross-attention, and feed-forward sub-layers,

- and shape-safe forward methods that accept sequence tensors and optional masks.

The main goal is not to match a specific library API, but to preserve the Transformer block structure and ensure every sub-layer leaves the tensor shape compatible with the next one.

Common mistakes

A common bug is applying layer normalization before the residual addition when the exercise expects post-norm behavior. Another is forgetting that the decoder's self-attention must be masked, which breaks autoregressive prediction. It is also easy to accidentally share one feed-forward network across all layers when the design expects each layer to have its own parameters. Finally, watch the tensor shape in the feed-forward network: it should operate position-wise, not flatten the entire sequence.

Residual wrapper shape check

Suppose a sub-layer preserves shape and returns a tensor of the same size as its input. The wrapper adds the original input back in and then normalizes, so the final output still matches the input shape.

```
x = np.zeros((2, 4, 8))
y = sublayer(x)
out = layer_norm(x + y)
assert out.shape == x.shape
```

Position-wise feed-forward on a sequence

A feed-forward block should transform each token independently while preserving the batch and time axes.

```
x = np.random.randn(3, 5, 16)
h = relu(x @ w1 + b1)
out = h @ w2 + b2
assert out.shape == (3, 5, 16)
```


Check your understanding

1. Which statement best describes why a residual connection is paired with layer normalization in a Transformer block?

- It helps preserve the original signal while making optimization of deep stacks easier.
- It replaces the need for attention weights.
- It forces every token to use the same attention pattern.
- It guarantees that all outputs become non-negative.

2. Explain how the decoder differs from the encoder in a Transformer block, focusing on the sub-layers and the masking behavior.

Sources: [4], [6]

Transformer mechanics check

Verify that you can trace tensor shapes and data flow through attention, masking, residual paths, normalization, and feed-forward sublayers in a full Transformer block.

In this checkpoint, you will demonstrate that you can trace how data moves through a full Transformer block: attention projections, masking, residual connections, layer normalization, and the position-wise feed-forward network. The questions focus on how shapes and dependencies change as tensors pass through encoder and decoder sub-layers, not on memorizing isolated definitions.

Check your understanding

1. A batch of encoder hidden states has shape $(\text{batch}, \text{seq_len}, \text{d_model}) = (32, 50, 512)$. A multi-head self-attention sub-layer with 8 heads and $\text{d}_k = \text{d}_v = 64$ is applied, followed by the output projection. Which statement best describes the shape of the sub-layer output that will feed the residual connection?

- $(32, 50, 512)$, because the concatenated head outputs are projected back to d_model .
- $(32, 50, 64)$, because each head produces 64 features and the heads are averaged.
- $(32, 8, 50, 64)$, because the head dimension is preserved all the way to the residual connection.

2. During decoder self-attention for position 7 in a target sequence, which behaviors must be true if the model is preserving autoregressive decoding? Select all that apply.

- Position 7 may attend to positions 1 through 7 in the decoder input.
- Position 7 may attend to position 8 if the softmax is computed on a masked score matrix.
- Illegal future positions are blocked by setting their attention logits to $-\infty$ before softmax.
- The decoder output embeddings are offset by one position to align known tokens with predictions.
- Every decoder position can freely attend to all positions in the encoder stack.

3. Complete the transformation used in a Transformer sub-layer with residual connection and normalization: _____

4. A decoder block receives tensors of shape $(\text{batch}, \text{tgt_len}, 512)$. In a few sentences, explain the order of operations through one decoder layer and why the shapes remain compatible from one sub-layer to the next.

Sources: [\[4\]](#), [\[5\]](#), [\[6\]](#)

MODULE 3

Training the translation model to match the paper

Length-based batching and training throughput

Form mini-batches by approximate sequence length and token counts to reproduce the paper's efficient GPU training schedule.

Objective

Learn how approximate sequence-length batching helps keep GPU training efficient by reducing padding and making token counts per batch more uniform.

Core Idea

When sentences in a mini-batch have similar lengths, less computation is wasted on padding tokens. That means the GPU spends more of its time on real data and less on empty slots. A common strategy is to group sentence pairs by approximate sequence length and then cap each batch by a target number of source and target tokens rather than by a fixed number of sentences. In one efficient training setup, each batch contained about 25,000 source tokens and 25,000 target tokens.[ed974866-1230-4c7d-9c1f-e986fbeb3596]

This matters because a batch of 32 very short sentences and a batch of 32 very long sentences do not cost the same amount to process. Token-based batching makes the workload more consistent from step to step, which helps stabilize training throughput.

Practical Example

Suppose you sort sentence pairs into buckets like short, medium, and long, then pack each mini-batch from one bucket at a time. A bucket of medium-length pairs might fill a batch with 24,800 source tokens and 24,900 target tokens, while another batch from the same bucket may have a similar total. The exact number of sentence pairs can change, but the token budget stays close to the target.

Common Misconception

A fixed batch size does **not** guarantee fixed training cost. If every batch contains the same number of sentences, a batch full of long sequences can be much slower than one full of short sequences. The more useful unit for throughput is often tokens, not examples.

Token-budgeted mini-batch

You have sentence pairs with token lengths (12, 14), (13, 11), (15, 16), (14, 13) for source and target. If you set a batch budget of about 50 source tokens and 50 target tokens, you can place all four pairs in one batch because the totals are $12 + 13 + 15 + 14 = 54$ source tokens and $14 + 11 + 16 + 13 = 54$ target tokens, which is close to the intended budget. If one pair were much longer, you would likely move it to a different bucket so the batch stays efficient.

Check your understanding

1. Which batching strategy is most likely to improve GPU training throughput for variable-length sentence pairs?

- Group sentence pairs by approximate length and keep a token budget per batch.
- Always use the same number of sentences in each batch, regardless of length.
- Randomly mix all lengths and only stop when the batch contains enough sentences.
- Pad every sequence to the maximum length in the entire dataset.

2. Explain why approximate length batching can make each training step faster and more predictable.

Sources: [9]

Optimizer schedule, label smoothing, and dropout

Match the paper's Adam settings, inverse-square-root learning-rate warmup, residual dropout, and label smoothing to stabilize translation training.

Objective

Learn how the Transformer's training recipe combines Adam, a warmup-based inverse-square-root learning-rate schedule, residual dropout, and label smoothing to make optimization stable and

translation quality better.

Core idea

The key idea is that these choices work together. Adam gives adaptive updates, but the learning rate still needs to change over time: it rises during warmup so early training does not become unstable, then decays like an inverse square root so later updates become more cautious. The paper's Adam settings are $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\epsilon = 10^{-9}$. The learning-rate rule is

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5})$$

which means the rate increases at first, then decreases after warmup.

Residual dropout is applied to each sub-layer output before it is added back to the input and normalized, and also to the sum of token embeddings and positional encodings. For the base model, the dropout rate is 0.1. Label smoothing with $\epsilon_{\text{ls}} = 0.1$ makes the target distribution less sharp, which can slightly hurt perplexity while improving accuracy and BLEU.

Practical illustration

If the model predicts a target word and the true label is softened, the training signal does not insist that only one class gets all the probability mass. That can help the model avoid becoming overconfident. For example, a hard one-hot target might be replaced by a distribution that assigns most probability to the correct token and a little to the others, which encourages smoother output probabilities.

Common misconception

A common mistake is to think label smoothing simply “makes the model worse” because it increases loss on the training set. In practice, a higher training loss can still correspond to better generalization, because the model learns calibrated, less brittle predictions rather than memorizing overly confident one-hot targets.

Reading the learning-rate schedule

Suppose d_{model} is fixed and training is in the warmup phase. Then the learning rate follows the $\text{step} \cdot \text{warmup}^{-1.5}$ part of the formula, so it grows as steps increase. After warmup, the $\text{step}^{-0.5}$ term dominates, so the learning rate decays gradually instead of staying constant.

Check your understanding

1. Why does the Transformer use a warmup-based inverse-square-root learning-rate schedule instead of a constant learning rate?

- It lets updates start cautiously, then decay later so training stays stable.
- It guarantees the training loss will always decrease every step.
- It removes the need for Adam by making the optimizer adaptive.
- It is only used so the model trains faster on the first batch.

2. Which training choices from the Transformer paper help regularize or stabilize translation training? Select all that apply.

- Residual dropout on sub-layer outputs and embedding-plus-position sums
- Label smoothing with $\epsilon_{\text{ls}}=0.1$
- Using a fixed learning rate with no warmup
- Adam with $\beta_1=0.9$, $\beta_2=0.98$, and $\epsilon=10^{-9}$
- Removing residual connections to reduce variance

3. In a few sentences, explain how warmup, residual dropout, and label smoothing each contribute differently to training a translation model.

Sources: [9], [10]

WMT data and subword preprocessing

Set up the WMT translation corpora with BPE-style subword tokenization and shared vocabularies so the training data matches the paper's regime.

Objective

Set up translation training data so it matches the regime used in the Transformer paper: use the WMT 2014 English–German or English–French corpora, segment text into subword units with BPE-style tokenization, and keep the source and target vocabulary aligned when appropriate.

[ed974866-1230-4c7d-9c1f-e986fbeb3596]

Core idea

Machine translation models do not usually learn one token at a time from raw words. Instead, long or rare words are broken into smaller, frequent pieces so the model can reuse patterns across many words. In the WMT setup described in the paper, English–German used byte-pair encoding with a shared source-target vocabulary of about 37,000 tokens, while English–French used a 32,000-piece vocabulary. [ed974866-1230-4c7d-9c1f-e986fbeb3596]

A shared vocabulary is useful when the two languages share many symbols, names, punctuation marks, and even some stems. It also keeps the preprocessing and the model’s embedding layer consistent: the same subword id can represent a piece on either side of the translation pair. That makes training closer to the paper’s regime and reduces out-of-vocabulary problems.

[dbb797a5-03f8-4c50-b775-8505db24dbed][ed974866-1230-4c7d-9c1f-e986fbeb3596]

Practical example

Suppose the word `internationalization` appears rarely in training. With subword preprocessing, it may be split into pieces like `inter@@`, `nation@@`, `al@@`, `ization` rather than treated as one unknown token. The model can then learn useful pieces once and combine them across many words. This is especially important in WMT-sized corpora, where coverage and efficiency matter.

[ed974866-1230-4c7d-9c1f-e986fbeb3596]

Common misconception

A common mistake is to think BPE is only about making the vocabulary smaller. Vocabulary size matters, but the bigger goal is better generalization: subwords let the model represent rare and unseen words compositionally instead of collapsing them into `<unk>`. Another misconception is that source and target vocabularies must always be separate; the paper’s English–German setup explicitly used a shared vocabulary. [ed974866-1230-4c7d-9c1f-e986fbeb3596]

From words to shared subwords

Take the English sentence A rare tokenized word appears and its German translation. If the corpus is tokenized with BPE, the rare word may be split into reusable pieces rather than kept whole. In a shared vocabulary, those pieces are assigned ids from one common lookup table, so the encoder and decoder can both map them through the same embedding matrix. [ed974866-1230-4c7d-9c1f-e986fb3596]

Check your understanding

1. If you are preparing WMT-style translation data for the Transformer paper, which preprocessing choice best matches the English–German setup?
 - Train on raw words and use a separate vocabulary for each language.
 - Apply BPE-style subword segmentation and use one shared source-target vocabulary.
 - Keep only the 1,000 most frequent full words and replace everything else with <unk>.
2. Explain why subword tokenization is helpful for WMT translation data, and mention one reason a shared vocabulary can be beneficial.

Sources: [9], [3], [7], [11]

Translation decoding and checkpoint averaging lab

Implement checkpoint averaging and beam-search decoding with length penalty and output-length limits, then verify that inference matches the reported setup.

Objective

In this activity, you will connect training-time model selection with inference-time decoding. The goal is to produce translations using an averaged checkpoint, then decode with beam search while respecting a length penalty and a maximum output-length cap. The reported setup matters: if your inference settings do not match the training report, your outputs can differ even when the model is otherwise correct. The original Transformer paper explicitly describes averaging the last checkpoints, using beam search with beam size 4, applying length penalty $\alpha = 0.6$, and limiting output length to input length + 50 while stopping early when possible. [f656ad30-554c-4544-ae4d-28dfe584edbf]

Core idea

Checkpoint averaging smooths out small fluctuations between recent saved parameter states. Instead of trusting a single checkpoint, you combine several recent ones into one set of weights, which often gives more stable predictions. At inference time, beam search explores multiple candidate sequences rather than greedily picking the most likely next token at each step. A length penalty adjusts scores so the decoder does not unfairly prefer very short outputs; this is especially important in translation, where the best answer is often a complete sentence rather than the shortest sentence.

A practical decoding loop usually does three things:

1. Scores next-token candidates for each partial hypothesis.
2. Keeps only the top few hypotheses according to beam score.
3. Stops when all beams have finished or when a length limit is reached.

Guided walkthrough

First, think about the checkpoint-averaging step as a preprocessing step for inference. Your code should accept multiple saved parameter snapshots, combine them into a single averaged parameter set, and use that result for decoding. This is not training; no gradients are involved.

Next, focus on beam search. Each partial translation keeps track of its token history, accumulated log-probability, and whether it has already produced an end-of-sequence token. After each decoding step, you expand the alive beams, score the candidates, and keep the best-scoring beams after applying the length penalty.

The length cap is not just a safety feature. It also defines when decoding must stop even if the model keeps proposing new tokens. The reported setup uses a cap based on source length, so your decoder should be able to reference the input length and stop when the generated length reaches the allowed maximum.

Finally, once decoding ends, the selected translation should come from the best finished hypothesis when available, not merely the beam with the highest unfinished score. That detail often changes the final output.

Exercise requirements

Your implementation should:

- Average a configurable list of checkpoints into one parameter set.
- Run beam-search decoding with a configurable beam size.
- Apply a length-penalized score when ranking hypotheses.
- Stop decoding when all beams finish or the maximum output length is reached.

- Use the averaged checkpoint for inference, and ensure the decoding settings match the reported configuration.

Common mistakes

- Averaging the wrong checkpoints, such as the earliest ones instead of the most recent ones.
- Applying the length penalty to raw probabilities instead of to the accumulated sequence score.
- Returning the best unfinished beam when a finished hypothesis exists.
- Ignoring the output-length cap, which can cause infinite or overly long generations.
- Forgetting that beam search rank-ordering should happen after each expansion, not only at the end.

Averaging recent checkpoints

Suppose you have three recent checkpoints whose parameter vectors are:

```
ckpts = [  
    {"w": 2.0, "b": 1.0},  
    {"w": 4.0, "b": 3.0},  
    {"w": 6.0, "b": 5.0},  
]
```

A checkpoint average would produce a single parameter set whose values are the elementwise mean across the chosen snapshots. This smoothed model is then used for inference instead of any one checkpoint alone.

Why length limits matter in decoding

Imagine a translation model that keeps assigning probability mass to continuing forever. A beam-search decoder with a source-length-based cap prevents unbounded generation and forces a decision within a known budget.

```
source_len = 12  
max_output_len = source_len + 50  
# Decoding must stop by this point, even if some beams are still alive.
```

That limit is part of the inference contract, not a training detail.

Check your understanding

1. When you average the last few checkpoints before inference, what is the main benefit?

- It reduces noise from short-term parameter fluctuations and often makes predictions more stable.
- It increases the beam size so the decoder explores more candidates.
- It converts greedy decoding into stochastic sampling.
- It guarantees that the model will always translate with perfect grammar.

2. Explain why a decoder should prefer a finished hypothesis when one is available, rather than simply returning the highest-scoring unfinished beam.

Sources: [12], [9]

MODULE 4

Experiments, ablations, and reproduction check

Constituency parsing reproduction

Examine the paper's WSJ parsing setup to understand how the Transformer transfers beyond translation and how its F1 compares to prior work.

Objective

Understand how the Transformer was evaluated on English constituency parsing, why this is a meaningful transfer test beyond translation, and how to interpret its WSJ F1 scores against earlier parsers.

Core idea

Constituency parsing asks a model to output a tree structure over a sentence, so the prediction is longer and more structurally constrained than the input. That makes it a good stress test for a sequence-to-sequence model built entirely on attention: if the model can learn parsing, it is not just memorizing translation-specific patterns. The paper trained a 4-layer Transformer with model dimension 1024 on the WSJ portion of Penn Treebank, using about 40K training sentences, and also tried a semi-supervised version with much larger extra corpora. The setup used limited tuning on Section 22 development data, then reported results on Section 23.

[8bdd2511-f47d-4b10-9795-03b3ad6a32ed]

How to read the parsing result table

A useful way to reproduce the experiment is to compare the Transformer to the strongest earlier WSJ-only parser and then to the semi-supervised group. In WSJ-only training, the Transformer reaches 91.3 F1, which is below Dyer et al. (2016) at 91.7 but above several older discriminative parsers. In the semi-supervised setting, the Transformer reaches 92.7 F1, which is better than the other listed semi-supervised systems but still below the best overall generative result of 93.3. That pattern suggests the model transfers well to parsing, though extra unlabeled or high-confidence data still helps.

[ab2073a0-bd06-4071-9be8-fc4896ee225c]

Check your understanding

1. If you are reproducing the WSJ-only parsing setup, which statement best describes the evaluation goal?

- Measure how well the Transformer generalizes from translation-style training to a structurally constrained output task.
- Tune a parser architecture until it reaches the best possible F1 on any parsing benchmark.
- Show that self-attention only works when the output sequence is shorter than the input.
- Demonstrate that the Transformer cannot handle non-translation tasks.

2. Which statements about the reported WSJ parsing setup and results are correct? Select all that apply.

- The WSJ-only Transformer used 4 layers and achieved 91.3 F1 on Section 23.
- The semi-supervised Transformer reached 92.7 F1 on Section 23.
- The paper reports that the Transformer beats every prior parser in both WSJ-only and semi-supervised settings.
- The semi-supervised setting used roughly 17M additional sentences and a 32K vocabulary.
- The Transformer was heavily re-tuned with a completely new architecture for parsing.

3. In one short paragraph, explain why the WSJ constituency parsing experiment is a strong test of the Transformer and what its results imply relative to earlier parsers.

Sources: [13], [14]

Transformer variant ablation study

Use the paper's Table 3 to read how head count, key dimension, width, depth, and dropout change perplexity and BLEU under fixed compute.

Objective

Learn how to read a Transformer ablation table to see which architectural choices matter most when compute is held roughly fixed.

Core idea

An ablation study changes one design choice at a time while keeping the rest of the model comparable. In Table 3, the authors compare head count, key/value dimension, width, depth, and dropout on the same translation development set, so you can interpret changes in perplexity and BLEU as effects of the architecture rather than of training data or evaluation setup [05f5f97d-91f2-4c70-9410-0cc1b7298906]. The main lesson is that better performance does not come from making every number bigger; it comes from balancing how attention, model width, and regularization interact under a compute budget.

What to look for in the table

Start with the base model, then compare each row group against it. If a change lowers perplexity and raises BLEU, it is usually an improvement; if both worsen, the change is probably harmful. For example, reducing the attention key size hurts quality, which suggests the model needs enough key/value space to compare tokens effectively [05f5f97d-91f2-4c70-9410-0cc1b7298906].

Practical example

Suppose you are choosing between two settings in row (A): 4 heads with smaller per-head vectors, or 16 heads with even smaller per-head vectors. The table shows that performance is not monotonic in the number of heads: too few heads can under-represent different relations, but too many heads can make each head too narrow to be useful [05f5f97d-91f2-4c70-9410-0cc1b7298906].

Common misconception

A common mistake is to assume that the biggest model or the most heads must always win. Table 3 shows the opposite can happen: moderate changes sometimes improve BLEU, but extreme settings can reduce it even when the compute budget is kept similar [05f5f97d-91f2-4c70-9410-0cc1b7298906].

That is why ablation studies are so useful: they reveal tradeoffs, not just trends.

Takeaway

When you read an ablation table, ask three questions: What was held constant? What changed? Did perplexity and BLEU move together or in opposite directions? That habit turns a table of numbers into evidence about which design choices actually matter.

Reading one row of the ablation table

Row (A) keeps total compute comparable while changing the number of heads and the per-head key/value dimensions. The key interpretation is that the best setting is not the one with the most heads: the table shows a middle ground can match or beat the base model, while a single head is clearly worse [05f5f97d-91f2-4c70-9410-0cc1b7298906]. In other words, head count and head size must be balanced, because shrinking each head too much can remove useful capacity even if the total work stays similar.

Check your understanding

1. In Table 3, what is the best general conclusion about changing the number of attention heads while keeping compute roughly fixed?

- More heads always improve BLEU because the model can attend to more positions.
- There is a tradeoff: too few heads can hurt, but too many heads can also hurt.
- The number of heads has no effect if the total parameter count is unchanged.
- Head count only matters for training speed, not translation quality.

2. Which changes in Table 3 are supported by the data as generally improving translation quality relative to weaker nearby settings? Select all that apply.

- Increasing the model width from 256 to 1024 in row (C)
- Reducing the attention key dimension too far
- Using an intermediate number of heads instead of a single head
- Removing dropout entirely in every setting
- Increasing model depth with enough regularization and width

Sources: [15]

Translation quality and generalization

Interpret the paper's translation results as evidence that the base and big models improve BLEU while remaining computationally efficient.

Objective

Interpret the Transformer's translation results as evidence that the **base** and **big** models improve BLEU while still being computationally efficient. The key claim is not just that the models translate well, but that they do so with a favorable quality-to-cost tradeoff.

[dd78601b-ba35-4bd1-acf5-8129917a6d2b][f656ad30-554c-4544-aead-28dfe584edbf]

Core Idea

BLEU measures how closely a system's translations match high-quality references, so higher BLEU usually means better translation quality. In the results table, the Transformer **base** model reaches strong BLEU scores on both English-to-German and English-to-French, and the Transformer **big** model improves them further. Just as important, these gains come with much lower training cost than many earlier systems, showing that better performance does not have to require dramatically more computation. [dd78601b-ba35-4bd1-acf5-8129917a6d2b][f656ad30-554c-4544-aead-28dfe584edbf]

Practical Example

Suppose you compare two models on English-to-German. One older model gets a lower BLEU score but needs far more FLOPs to train, while the Transformer base model gets a higher BLEU score at a fraction of that cost. That means the new model is not only more accurate, but also more practical to train and scale. [dd78601b-ba35-4bd1-acf5-8129917a6d2b]

A small way to think about this is:

$\text{\texttt{\text{better model}}} = \text{\texttt{\text{higher BLEU}}} + \text{\texttt{\text{lower training cost}}}$

Of course, real model choice involves more than one metric, but this captures why the paper's translation results are notable.

Misconception

A common mistake is to assume that a higher BLEU score must always come from a much larger or more expensive model. The Transformer results show the opposite pattern: the base model is already competitive, and the big model pushes BLEU even higher while still remaining efficient relative to older approaches. [dd78601b-ba35-4bd1-acf5-8129917a6d2b][f656ad30-554c-4544-aead-28dfe584edbf]

Reading the translation table

The table compares BLEU and training cost across systems. The Transformer base model reaches 27.3 BLEU on English-to-German with a training cost of about $3.3 \cdot 10^{18}$ FLOPs, while the Transformer big model reaches 28.4 BLEU with about $2.3 \cdot 10^{19}$ FLOPs. This shows two things at once: quality improves from base to big, and both models remain far more efficient than several earlier high-performing systems.

Check your understanding

1. Which statement best captures the paper's translation result?

- The Transformer base and big models improve BLEU while using less training compute than many previous systems.
- Higher BLEU only appears when a model is trained with much larger computational cost.
- The Transformer models are efficient, but their BLEU scores are worse than older methods.
- Training cost is the only metric that matters, so BLEU can be ignored.

2. In 2–4 sentences, explain why the base and big Transformer models are a strong result for machine translation.

3. Which conclusions are supported by the translation results? Select all that apply.

- The big model improves BLEU over the base model.
- The base model already surpasses many previously published models.
- The models are only impressive because they use the largest training budgets in the table.
- The results suggest better generalization, since the models perform strongly on unseen test sets.
- The paper shows that translation quality and computational efficiency can improve together.

Sources: [12], [10]

Reproduce paper experiments check

Verify that you can explain the main ablations, the translation outcomes, and the parsing generalization results from the paper.

In this checkpoint, you will demonstrate that you can interpret the paper's main experimental results as a coherent story: which architectural choices mattered in the ablation study, how the translation systems compared to prior work, and what the parsing results say about generalization. Focus on reading the tables as evidence, not memorizing isolated numbers.

Check your understanding

1. A team is choosing a Transformer configuration for an English-to-German system and wants the best tradeoff between model quality and computational cost. Based on the ablation results, which change is most clearly associated with the strongest improvement over the base configuration when capacity is increased substantially?

- Reduce the model size from the base setting to a much smaller model with fewer parameters.
- Increase the model width and feed-forward size, as in the large 'big' configuration.
- Replace dropout with no regularization everywhere to maximize training fit.
- Use single-head attention instead of multi-head attention to simplify the architecture.

2. You are comparing the paper's translation results against previous systems and trying to state the most defensible conclusions. Which of the following statements are supported by the reported results?

- The Transformer base model beats all previously published single-model EN-DE results in Table 2.
- The Transformer big model achieves a new best EN-DE BLEU score of 28.4.
- The Transformer big model is trained at the highest cost in the table by a wide margin.
- The Transformer big model also performs very competitively on EN-FR, reaching 41.8 BLEU.
- The paper claims the translation gains come only from beam search and not from the model architecture.

3. In the parsing experiment, the Transformer was evaluated on WSJ Section ____.

4. In a brief paragraph, explain what the parsing results show about the Transformer's ability to generalize beyond translation. Mention the comparison to prior work and what stands out about the supervised and semi-supervised settings.

Sources: [15], [14], [12], [10]

References

[1] 1706.03762v7.pdf, Page 2, p. 2

ra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22]. End-to-end memory networks are based on a recurrent attention mechanism instead of sequence- aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [34]. To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence- aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [17, 18] and [9].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $z = (z_1, \dots, z_n)$. Given z , the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next. 2

[2] 1706.03762v7.pdf, Page 2, p. 2

output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network. In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs. 2

Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2. Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of

[3] 1706.03762v7.pdf, Page 1, p. 1

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works. Attention Is All You Need Ashish Vaswani¹ Google Brain avaswani@google.com Noam Shazeer² Google Brain noam@google.com Niki Parmar¹ Google Research nikip@google.com Jakob Uszkoreit² Google Research usz@google.com Llion Jones¹ Google Research llion@google.com Aidan N. Gomez³ ¹ University of Toronto aidan@cs.toronto.edu ² Lukasz Kaiser¹ Google Brain lukaszkaizer@google.com ³ Illia Polosukhin¹ ¹ illia.polosukhin@gmail.com

Abstract The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English- to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the

[4] 1706.03762v7.pdf, Page 3, p. 3

Figure 1: The Transformer - model architecture. The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions

[5] 1706.03762v7.pdf, Page 5, p. 5

output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2. Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this. $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$ where $\text{head}_i = \text{Attention}(QW^{Q_i}, KW^{K_i}, VW^{V_i})$ Where the projections are parameter matrices $W^{Q_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^{K_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^{V_i} \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$. In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in

[6] 1706.03762v7.pdf, Page 5, p. 5

the decoder, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between. $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$ (2) While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_f = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our

[7] 1706.03762v7.pdf, Page 2, p. 2

1 Introduction Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15]. Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t , as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains. Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network. In this work we propose the Transformer, a model architecture

[8] 1706.03762v7.pdf, Page 6, p. 6

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention. Layer Type Complexity per Layer Sequential Maximum Path Length Operations Self-Attention $O(n^2 \cdot d)$ $O(1)$ $O(1)$ Recurrent $O(n \cdot d^2)$ $O(n)$ $O(n)$ Convolutional $O(k \cdot n \cdot d^2)$ $O(1)$ $O(\log k(n))$ Self-Attention (restricted) $O(r \cdot n \cdot d)$ $O(1)$ $O(n/r)$ 3.5 Positional Encoding Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9]. In this work, we use sine and cosine functions of different frequencies: $P E(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$ $P E(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d_{\text{model}}})$ where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , $P E_{\text{pos}+k}$ can be represented as a

[9] 1706.03762v7.pdf, Page 7, p. 7

tention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source- target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [38]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula: $\text{lrate} = d^{-0.5} \cdot \min(\text{step_num}^{-0.5},$

[10] 1706.03762v7.pdf, Page 8, p. 8

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU	Training Cost (FLOPs)					
EN-DE	EN-FR	EN-DE	EN-FR				
ByteNet [18]	23.75	Deep-Att + PosUnk [39]	39.2	$1.0 \cdot 10^{20}$			
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$			
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$			
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$			
Deep-Att + PosUnk Ensemble [39]	40.4	8.0	10^{20}	GNMT + RL Ensemble [38]	26.30		
41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$	ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$
Residual Dropout	We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{\text{drop}} = 0.1$.						

Label Smoothing During training, we employed label smoothing of value $\epsilon = 0.1$ [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our

[11] 1706.03762v7.pdf, Page 5, p. 5

and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [30]. In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

[12] 1706.03762v7.pdf, Page 8, p. 8

by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models. On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{\text{drop}} = 0.1$, instead of 0.3. For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38]. Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU 5. 6.2 Model Variations To evaluate the importance of different components of the Transformer, we varied our

[13] 1706.03762v7.pdf, Page 9, p. 9

ality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model. 6.3 English Constituency Parsing To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37]. We trained a 4-layer transformer with $d_{\text{model}} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting. We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we 9

[14] 1706.03762v7.pdf, Page 10, p. 10

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ) Parser Training WSJ 23 F1 Vinyals & Kaiser et al. (2014) [37] WSJ only, discriminative 88.3 Petrov et al. (2006) [29] WSJ only, discriminative 90.4 Zhu et al. (2013) [40] WSJ only, discriminative 90.4 Dyer et al. (2016) [8] WSJ only, discriminative 91.7 Transformer (4 layers) WSJ only, discriminative 91.3 Zhu et al. (2013) [40] semi-supervised 91.3 Huang & Harper (2009) [14] semi-supervised 91.3 McClosky et al. (2006) [26] semi-supervised 92.1 Vinyals & Kaiser et al. (2014) [37] semi-supervised 92.1 Transformer (4 layers) semi-supervised 92.7 Luong et al. (2015) [23] multi-task 93.0 Dyer et al. (2016) [8] generative 93.3 increased the maximum output length to input length + 300. We used a beam size of 21 and $\beta = 0.3$ for both WSJ only and the semi-supervised setting. Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [8]. In contrast to RNN sequence-to-sequence models [37], the Transformer outperforms the Berkeley-Parser [29] even when training only on the WSJ training set of 40K sentences. 7 Conclusion In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention. For translation tasks, the Transformer can be trained

[15] 1706.03762v7.pdf, Page 9, p. 9

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities. N d model dff h d k dv Pdrop $\square$ ls train PPL BLEU params steps (dev) (dev) $\times 10^6$ base 6 512 2048 8 64 64 0.1 0.1 100K 4.92 25.8 65 (A) 1 512 512 5.29 24.9 4 128 128 5.00 25.5 16 32 32 4.91 25.8 32 16 16 5.01 25.4 (B) 16 5.16 25.1 58 32 5.01 25.4 60 (C) 2 6.11 23.7 36 4 5.19 25.3 50 8 4.88 25.5 80 256 32 32 5.75 24.5 28 1024 128 128 4.66 26.0 168 1024 5.12 25.4 53 4096 4.75 26.2 90 (D) 0.0 5.77 24.6 0.2 4.95 25.5 0.0 4.67 25.3 0.2 5.47 25.7 (E) positional embedding instead of sinusoids 4.92 25.7 big 6 1024 4096 16 0.3 300K 4.33 26.4 213 development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3. In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads. In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as